

Department of Computer Engineering
Academic Term: July-November 2022

Class : B.E. Computer
Semester : VII

Subject Name :ML
Subject Code : CSC701

Practical No:	7
Title.	To Study and implement Ensemble Techniques
Date of Performance.	19/09/2025
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation

Indicator	Very Poor	Poor	Average	Good	Excellent
Timeline (2)	More than three sessions late (0)	More than two sessions late (0.5)	Two sessions late (1)	One session late (1.5)	Early or ontime (2)
Efforts(4)	N/A	N/A	Not Completed (1)	Partially Completed (2)	Completed(4)
Legibility(2)	N/A	N/A	Poor(1)	Good(1.5)	Very Good(2)
Oral Assessment (2)	N/A	N/A	N/A	Partially Understood the concept (1)	Understood the concept(2)

```
In [ ]: # Import pandas Library
import pandas as pd
from matplotlib.colors import ListedColormap
import numpy as nm
import matplotlib.pyplot as mtp
from matplotlib.colors import ListedColormap
# initialize list of lists
data = [[3,1,1], [3,-1,1], [6,1,1], [6,-1,1], [1,0,0], [0,1,0], [0,-1,0], [-1,0,0]]
data1 = [[2,2,1], [2,-2,1], [-2,-2,1], [-2,2,1], [1,1,0], [1,-1,0], [-1,-1,0], [-1,1,0]]
data2 = [[1,1,1], [2,2,1], [2,0,1],[0,0,0], [-1,0,0], [0,1,0]]

# Create the pandas DataFrame
df = pd.DataFrame(data, columns=['Col1', 'Col2', 'Output'])
df1 = pd.DataFrame(data1, columns=['Col1', 'Col2', 'Output'])
df2 = pd.DataFrame(data2, columns=['Col1', 'Col2', 'Output'])
# print dataframe.
df
```

```
Out[ ]:
```

	Col1	Col2	Output
0	3	1	1
1	3	-1	1
2	6	1	1
3	6	-1	1
4	1	0	0
5	0	1	0
6	0	-1	0
7	-1	0	0

```
In [ ]: def plot_svm(df) :
#Extracting Independent and dependent Variable
x= df.iloc[:, [0,1]].values
y= df.iloc[:, 2].values
# Splitting the dataset into training and test set.
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.25, random_state=0)
#feature Scaling
from sklearn.preprocessing import StandardScaler
st_x= StandardScaler()
x_train= st_x.fit_transform(x_train)
x_test= st_x.transform(x_test)
from sklearn.svm import SVC # "Support vector classifier"
classifier = SVC(kernel='linear', random_state=0)
classifier.fit(x_train, y_train)
#Predicting the test set result
y_pred= classifier.predict(x_test)
#Creating the Confusion matrix
from sklearn.metrics import confusion_matrix
cm= confusion_matrix(y_test, y_pred)
x_set, y_set = x_train, y_train
x1, x2 = nm.meshgrid(nm.arange(start = x_set[:, 0].min() - 1, stop = x_set[:, 0].max() + 1, step = 0.5),
nm.arange(start = x_set[:, 1].min() - 1, stop = x_set[:, 1].max() + 1, step = 0.5))
mtp.contourf(x1, x2, classifier.predict(nm.array([x1.ravel(), x2.ravel()]).T).reshape(x1.shape),
alpha = 0.75, cmap = ListedColormap(('red', 'green')))
mtp.xlim(x1.min(), x1.max())
mtp.ylim(x2.min(), x2.max())
```

```

for i, j in enumerate(nm.unique(y_set)):
    mtp.scatter(x_set[y_set == j, 0], x_set[y_set == j, 1],
                c = ListedColormap(('red', 'green'))(i), label = j)
mtp.title('SVM classifier (Training set)')
mtp.xlabel('Col1')
mtp.ylabel('Col2')
mtp.legend()
mtp.show()

#Visulaizing the test set result

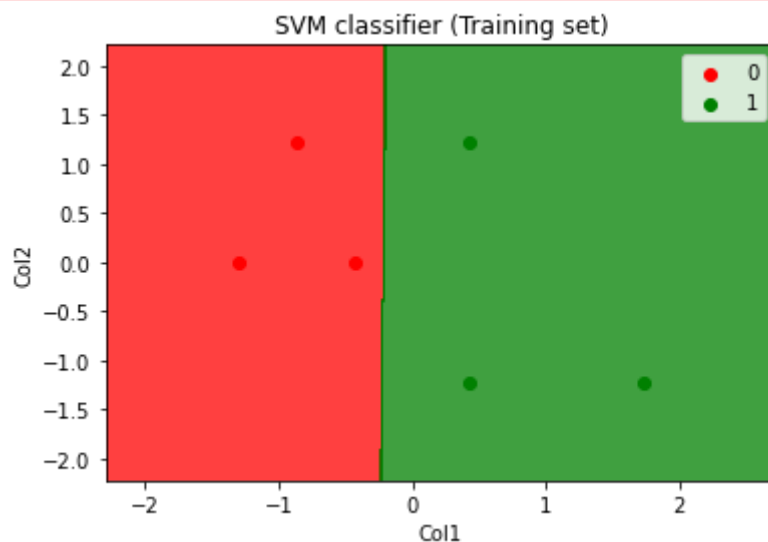
x_set, y_set = x_test, y_test
x1, x2 = nm.meshgrid(nm.arange(start = x_set[:, 0].min() - 1, stop = x_set[:, 0].max() + 1, step = 0.5),
                     nm.arange(start = x_set[:, 1].min() - 1, stop = x_set[:, 1].max() + 1, step = 0.5))
mtp.contourf(x1, x2, classifier.predict(nm.array([x1.ravel(), x2.ravel()]).T).reshape(x1.shape),
             alpha = 0.75, cmap = ListedColormap(('red', 'green' )))
mtp.xlim(x1.min(), x1.max())
mtp.ylim(x2.min(), x2.max())
for i, j in enumerate(nm.unique(y_set)):
    mtp.scatter(x_set[y_set == j, 0], x_set[y_set == j, 1],
                c = ListedColormap(('red', 'green'))(i), label = j)
mtp.title('SVM classifier (Test set)')
mtp.xlabel('Col1')
mtp.ylabel('Col2')
mtp.legend()
mtp.show()

```

In []: plot_svm(df)

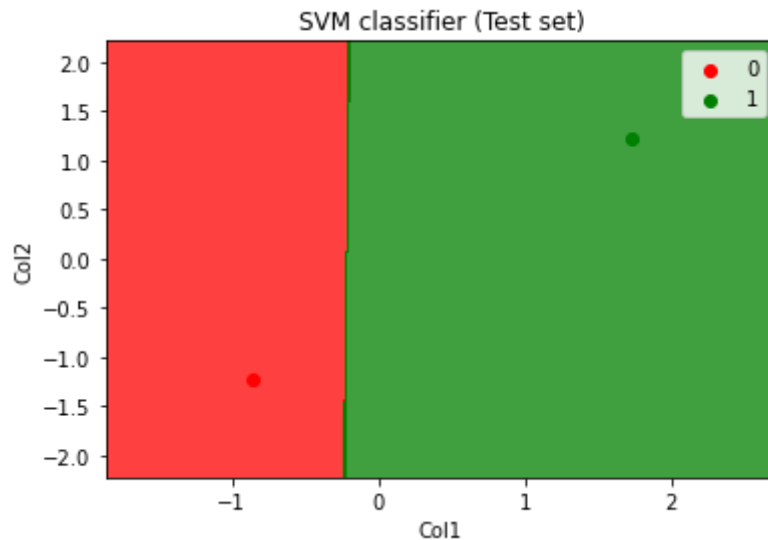
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2-D array with a single row if you intend to specify the same RGB or RGBA value for all points.

WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2-D array with a single row if you intend to specify the same RGB or RGBA value for all points.



WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2-D array with a single row if you intend to specify the same RGB or RGBA value for all points.

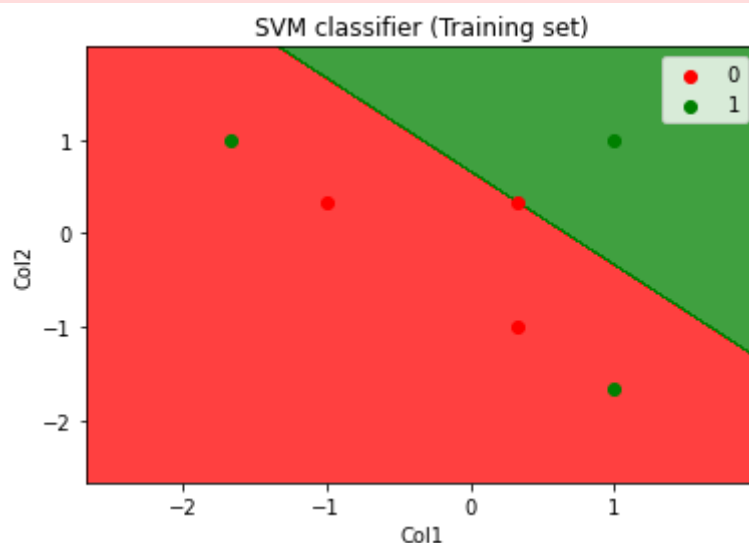
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2-D array with a single row if you intend to specify the same RGB or RGBA value for all points.



```
In [ ]: plot_svm(df1)
```

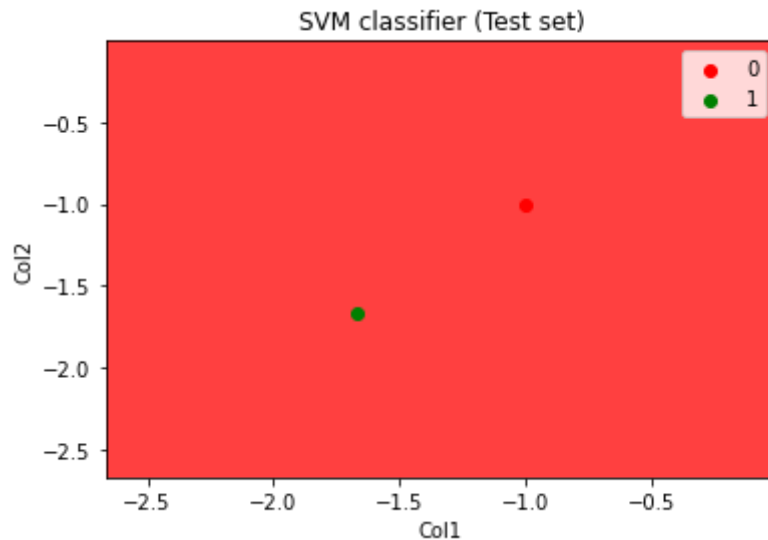
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2-D array with a single row if you intend to specify the same RGB or RGBA value for all points.

WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2-D array with a single row if you intend to specify the same RGB or RGBA value for all points.



WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2-D array with a single row if you intend to specify the same RGB or RGBA value for all points.

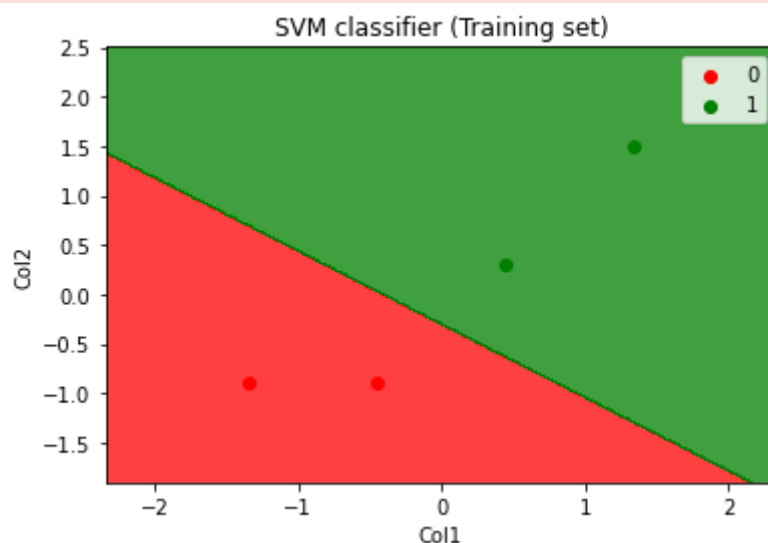
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2-D array with a single row if you intend to specify the same RGB or RGBA value for all points.



```
In [ ]: plot_svm(df2)
```

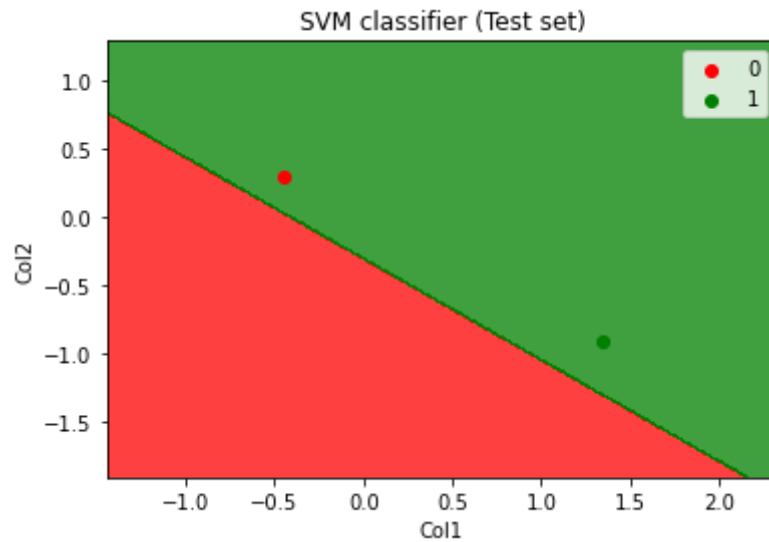
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2-D array with a single row if you intend to specify the same RGB or RGBA value for all points.

WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2-D array with a single row if you intend to specify the same RGB or RGBA value for all points.



WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2-D array with a single row if you intend to specify the same RGB or RGBA value for all points.

WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2-D array with a single row if you intend to specify the same RGB or RGBA value for all points.



Q.1 What is geometric intuition behind SVM?

→ The main intention is to find ~~a~~ the optimal boundary - hyperplane, that is the best line which separates data points of classes while maximizing margin between them.

Q.2 How would you explain convex hull in light of SVM

→ We simply build a convex hull for class A and B and draw a perpendicular on the shortest distance between closest points of both these hulls.

Q.3 What do you know about hard margin SVM and soft margin SVM?

→ If a point a satisfies the equation $(w \cdot a + b) \geq 1$ then a is correctly classified. So if the points are linearly separable then only the hyperplane is able to distinguish between them.

Q.4 What is hinge loss?

→ Hinge loss is a loss function which penalizes the SVM model for inaccurate prediction.

If $y_i(w \cdot a + b) \geq 1$, hinge loss is '0' i.e. points are correctly classified, when $y_i(w \cdot a + b) < 1$, hinge loss increases massively.

As $y_i(w \cdot a + b)$ increases with every misclassified point, the upper bound of hinge loss $\{1 - y_i(w \cdot a + b)\}$ also increases exponentially.